DateTime::diff()), and from_string (true if the interval was created with DateInterval::createFromString()) are also available. DateInterval::format() will return a formatted string in the same way as DateTimeInterface::format() but with a slight difference – DateInterval::format() requires that each format designator requires a % symbol in front of it (a little like a printf() formatting string).

```
print $dti->format("Y-m-d H:i:s") . PHP_EOL;
print $dti->format("%Y-%m-%d %H:%i:%s");
```

```
Output:
Y-m-d H:i:s
00-0-2 05:0:0
```

The DatePeriod class is in the same ballpark as DateInterval, but its purpose is slightly different. DatePeriod is intended for use in iterative code: it defines an interval of time that needs to elapse per repetition, a start time, and an end time. Between the start and end, every time an interval has elapsed, it triggers an iteration. DatePeriod implements the IteratorAggregte interface, which allows it to be used as an array in control structures like foreach.

```
$period = new DatePeriod(
    $bellRiots,
    new DateInterval('PT12H'),
    $retakeSanctuary
);

foreach ($period as $date) {
    echo $date->format(DateTimeImmutable::COOKIE) . PHP_EOL;
}
```

```
Output:
Sunday, 01-Sep-2024 00:00:00 PST
Sunday, 01-Sep-2024 12:00:00 PST
Monday, 02-Sep-2024 00:00:00 PST
Monday, 02-Sep-2024 12:00:00 PST
Tuesday, 03-Sep-2024 00:00:00 PST
```